

Para ejecutar este experimento real en tu laboratorio de **4 equipos de hardware**, necesitas implementar una **estrategia de marcas de tiempo sincronizadas** (timestamps). Como los datos viajan entre ordenadores distintos, no puedes usar el reloj interno de cada máquina de forma aislada (porque si un portátil va 2 segundos adelantado respecto a la Raspberry Pi, los cálculos de latencia saldrán negativos o erróneos).

A continuación, tienes la guía paso a paso de **cómo programar y ejecutar la medición exacta** de las tres etapas de forma práctica:

Paso 0: Sincronizar los relojes (Obligatorio)

Antes de encender los scripts, los 3 equipos de la capa Fog (2 Raspberry Pi y el Portátil Validador) y el Portátil Edge (el de las 11 simulaciones) deben sincronizarse al mismo milisegundo por red local.

- **Cómo se hace:** Configura uno de tus portátiles como servidor NTP local y haz que las Raspberry Pi sincronicen su reloj contra él, o fuerza la sincronización a través de internet ejecutando en la terminal de Linux:

```
sudo timedatectl set-ntp true
```

Paso 1: Modificar los Scripts para Capturar los Datos

Debes añadir variables que guarden la hora exacta en **milisegundos Unix** a medida que el dato viaja por la arquitectura.

A. En el Portátil de las 11 Simulaciones y el ESP32-C3 real:

En el código Python (versión 3.11) que simula los sensores, justo antes de empaquetar el mensaje CoAP, captura el tiempo de origen y mételo **dentro del propio cuerpo del JSON/Payload** que vas a enviar:

```
# Script simulador en el Portátil Edge
import time
import json

# Captura el tiempo exacto en milisegundos
t_origen = int(time.time() * 1000)

# El payload lleva el eCO2 y el sello de tiempo de nacimiento del
dato
```

```
payload = {
    "co2": 450,
    "t_origen": t_origen
}
# Enviar por CoAP hacia la Raspberry Pi
```

B. En las 2 Raspberry Pi 4 (Servidores CoAP):

Modifica el método `render_post(request)` de tu servidor CoAP para que capture el tiempo en dos momentos clave:

```
# Servidor CoAP en la Raspberry Pi 4
import time

async def render_post(request):
    # Momento 1: El datagrama UDP acaba de entrar por red
    t_llegada_red = int(time.time() * 1000)

    data = json.loads(request.payload)
    t_origen = data["t_origen"]

    # CÁLCULO ETAPA 1 (LATENCIA DE RED NETO)
    latencia_red = t_llegada_red - t_origen

    # ... El script acumula las 5 muestras en el búfer ...
    # ... Justo antes de llamar a la función de la Blockchain:

    t_salida_fog = int(time.time() * 1000)

    # CÁLCULO ETAPA 2 (PROCESAMIENTO INTERNO EN EL FOG)
    latencia_fog = t_salida_fog - t_llegada_red

    # Pasamos los datos al conector Web3 junto con los cálculos
    # parciales
    await enviar_a_blockchain(promedio, t_origen, latencia_red,
    latencia_fog)
```

C. En el Portátil Validador (Conector Web3 / Geth):

Cuando la tarea asíncrona envía la transacción con `transact()` y espera a que se mine el bloque con el recibo, calculas la etapa final:

```
# Conector Web3 en Python
```

```

async def enviar_a_blockchain(promedio, t_origen, latencia_red,
latencia_fog):
    tx_hash =
contrato.functions.reportarCO2(promedio).transact({'from':
CUENTA_FOG})

    # Espera asíncrona a que el bloque se mine bajo el consenso
Clique
    recibo = await async_wait_for_transaction_receipt(tx_hash)

    # Momento 3: El bloque ha sido validado e incluido en la
blockchain
    t_confirmacion_blockchain = int(time.time() * 1000)

    # CÁLCULO ETAPA 3 (TIEMPO DE CONSENSO Y MINADO DE LA EVM)
    # Restamos el tiempo acumulado anterior para aislar la etapa
blockchain
    latencia_blockchain = t_confirmacion_blockchain - (t_origen +
latencia_red + latencia_fog)

    # Guardamos los tres datos limpios en un archivo CSV local para
graficar después
    with open("resultados_latencia.csv", "a") as f:

f.write(f"{latencia_red},{latencia_fog},{latencia_blockchain}\n")

```

Paso 2: Cómo Ejecutar el Experimento de Carga

Para obtener los datos de la gráfica de áreas apiladas, tienes que variar la carga del sistema (el número de nodos concurrentes) ejecutando **cuatro pruebas distintas** de al menos 5 minutos cada una:

1. **Prueba de Carga Baja (1 Nodo):** Enciende únicamente el ESP32-C3 físico. Deja que envíe datos y guarda el CSV.
2. **Prueba de Carga Media (4 Nodos):** Enciende el ESP32-C3 físico y activa solo 3 de las 11 simulaciones en el portátil Edge.
3. **Prueba de Carga Alta (8 Nodos):** Enciende el ESP32-C3 físico y activa 7 simulaciones en el portátil Edge.
4. **Prueba de Carga Máxima (12 Nodos):** Enciende el ESP32-C3 físico y activa las 11 simulaciones simultáneamente en el portátil Edge.

Al final de las cuatro pruebas, abres el archivo resultados_latencia.csv, calculas la media aritmética de cada etapa para los escenarios de 1, 4, 8 y 12 nodos, y **esos 4 números promedios son los que introduces en los arrays del código de Matplotlib** (latencia_red,

latencia_fog, latencia_blockchain) para pintar la gráfica definitiva.

Paso 3: El Enfoque Comparativo (Edge-to-Cloud directo)

Para demostrar al tribunal que tu modelo Edge-to-Fog es superior, tienes que hacer una **quinta prueba de contraste** simulando el entorno clásico sin la capa intermedia:

- **Cómo se ejecuta:** Modifica temporalmente el script de simulación del portátil Edge para que **no envíe los datos por CoAP a las Raspberry Pi**. En su lugar, haz que los 11 simuladores ejecuten directamente llamadas HTTP RPC utilizando la librería web3.py para atacar directamente al portátil validador de la Blockchain.
- **El Resultado:** Verás que, al no haber un búfer en el Fog que agrupe los datos de 5 en 5, el portátil validador recibe una ráfaga masiva de transacciones individuales por segundo. Las peticiones HTTP empezarán a acumularse en cola, el tiempo de espera del recibo se disparará y la latencia E2E escalará exponencialmente, demostrando empíricamente por qué tu arquitectura cooperativa es necesaria.

¿Ha quedado más claro el flujo de ejecución y cómo capturar los tiempos en los scripts? Si estás listo, podemos pasar a estructurar la **Sección 6.3 dedicada al uso de CPU y RAM con psutil en las Raspberry Pi 4**.